Note that none of these examples rely on a particular UI. If these rules are only ever calculated on the server side, they could be tested without having to manipulate the UI at all. This is often the case with acceptance criteria around business rules.

EXERCISE 9.1 The credit card eligibility table in the previous example is incomplete. For example, it doesn't explore examples of eligibility that depend on age or job history. Complete the preceding scenario with additional business rules until you think you have enough to implement a solution.

Discovering examples through conversation

As you've seen, conversation is the cornerstone of BDD, but not all conversations are equal. BDD conversations are more valuable when they're discovering new examples and expanding the team's collective understanding. In practice, the time a team has to work through and clarify acceptance criteria isn't infinite, and this process monopolizes the time and attention of several team members. It's important to focus first on high-value business rules and on areas of uncertainty, and then on features that are better known and less risky.

For example, suppose you're working on a story card related to authentication. If your application uses an existing, well-known authentication mechanism that the team has already worked with, the acceptance criteria for the related business rules can afford to be succinct. You might have a basic scenario such as the following:

```
Given Joe has a valid LDAP account
And Joe has application permissions for application X
When Joe logs on
Then Joe should be given access to the application
```

To this you might add a few counterexamples: invalid username or password, insufficient permissions, and so forth. But if this is familiar territory for the team, there's little benefit spending a lot of time working through these examples together, and the developers may be able to implement the feature using only a small set of examples.

On the other hand, if the application uses a new authentication strategy or technology, it will be valuable to spend more time working through the unknowns. Where are user accounts stored? Will there be single-sign-on between this application and other applications within the organization? What application-specific permissions need to be managed, and who will manage them? And so on.

One of the big benefits of BDD comes from the way conversation can be used to reveal assumptions and flush out uncertainty, expanding your collective understanding of a problem domain. Naturally, this process is more beneficial where there are unknowns to be flushed out, and the law of diminishing returns is very applicable. As a rule of thumb, if you're in a Three Amigos session and the scenarios start to feel like you're "stating the obvious," it's probably time to move on to another requirement.

In fact, although you'll need to have UI-based acceptance criteria that illustrate how users will interact with the application, and to demonstrate and verify any business logic that's implemented within the UI layer itself, acceptance criteria that relate to